

Optimization in Code: Derivatives You Can Trust

Week 1 Seminar: matrix calculus, gradient checks, and regression

A. D. Ignatov
HSE University, Faculty of Computer Science

2026–2027

Today we will build a derivative-testing workflow

By the end of the seminar, you will be able to:

- ① translate a scalar objective into vectorized NumPy code;
- ② derive gradients of quadratic, linear-regression, and logistic losses;
- ③ classify stationary points using Hessian eigenvalues;
- ④ implement central-difference gradient checking;
- ⑤ detect shape, scaling, sign, and numerical-stability bugs.

Takeaway. A gradient is not finished when it is derived; it is finished when it passes an independent check.

Warm-up: read the geometry before differentiating

For each objective, predict the stationary points and their type.

$$f_1(x, y) = x^2 + 4y^2$$

What do its contours look like?

$$f_2(x, y) = x^2 - y^2$$

Can a zero gradient guarantee a minimum?

$$f_3(x) = (x^2 - 1)^2$$

How many local and global minima?

Question. Which information is supplied by the gradient, and which by the Hessian?

Shapes are part of the mathematics

We use column-vector mathematics and one-dimensional NumPy arrays:

$$X \in \mathbb{R}^{n \times d}, \quad w \in \mathbb{R}^d, \quad y \in \mathbb{R}^n, \quad Xw \in \mathbb{R}^n.$$

```
X.shape      # (n, d)
w.shape      # (d,)
y.shape      # (n,)
(X @ w).shape # (n,)
```

Common bugs:

- accidental $(n, 1)$ arrays and unwanted broadcasting;
- $X @ X.T$ instead of $X.T @ X$;
- summing the loss but differentiating its mean;
- confusing elementwise $*$ with matrix multiplication $@$.

Differentials provide a repeatable derivation recipe

To obtain $\nabla f(x)$:

- ① write df using product and chain rules;
- ② move every dx to the final position;
- ③ rewrite $df = \langle g, dx \rangle$;
- ④ read off $g = \nabla f(x)$;
- ⑤ verify the shape of g matches the shape of x .

Useful identities:

$$\begin{aligned}d(Ax) &= A dx, & d(x^\top y) &= y^\top dx + x^\top dy, \\d(x^\top Ax) &= \langle (A + A^\top)x, dx \rangle.\end{aligned}$$

Exercise 1: derive a quadratic gradient

Let

$$f(x) = \frac{1}{2}x^\top Ax + b^\top x + c, \quad A \in \mathbb{R}^{d \times d}.$$

- ① Find $\nabla f(x)$ without assuming that A is symmetric.
- ② Find $H_f(x)$.
- ③ Which part of A influences $x^\top Ax$?
- ④ Under what condition is the stationary point unique?

Do not expand the expression coordinate by coordinate. Use differentials.

Exercise 1: solution and implementation

$$\nabla f(x) = \frac{1}{2}(A + A^\top)x + b, \quad H_f(x) = \frac{1}{2}(A + A^\top).$$

Only the symmetric part $\frac{1}{2}(A + A^\top)$ contributes because the skew-symmetric part satisfies $x^\top (A - A^\top)x = 0$.

```
def quadratic_value(x, A, b, c=0.0):  
    return 0.5 * x @ A @ x + b @ x + c  
  
def quadratic_grad(x, A, b):  
    return 0.5 * (A + A.T) @ x + b  
  
def quadratic_hessian(A):  
    return 0.5 * (A + A.T)
```

A Hessian classifier needs tolerance, not exact zero

```
import numpy as np

def classify_hessian(H, tol=1e-8):
    eig = np.linalg.eigvalsh(0.5 * (H + H.T))
    if np.all(eig > tol):
        return "strict local minimum"
    if np.all(eig < -tol):
        return "strict local maximum"
    if eig.min() < -tol and eig.max() > tol:
        return "saddle point"
    return "second-order test is inconclusive"
```

Why eigvalsh rather than eigvals? Why symmetrize H numerically?

Exercise 2: classify four stationary points

Classify $x = 0$ using derivatives up to second order:

① $f_1(x, y) = x^2 + 2y^2;$

② $f_2(x, y) = x^2 - y^2;$

③ $f_3(x, y) = x^4 + y^4;$

④ $f_4(x, y) = x^3 + y^2.$

For inconclusive cases, use the definition or inspect carefully chosen directions.

Exercise 2: what the Hessian can and cannot decide

Function	$\nabla f(0)$	$H_f(0)$	Conclusion
$x^2 + 2y^2$	0	$\text{diag}(2, 4)$	strict minimum
$x^2 - y^2$	0	$\text{diag}(2, -2)$	saddle
$x^4 + y^4$	0	0	minimum; test inconclusive
$x^3 + y^2$	0	$\text{diag}(0, 2)$	saddle; test inconclusive

For f_4 , restrict to $y = 0$:

$f_4(x, 0) = x^3$ takes both signs near zero.

Takeaway. A semidefinite Hessian does not settle the question when flat directions remain.

Central differences independently estimate gradients

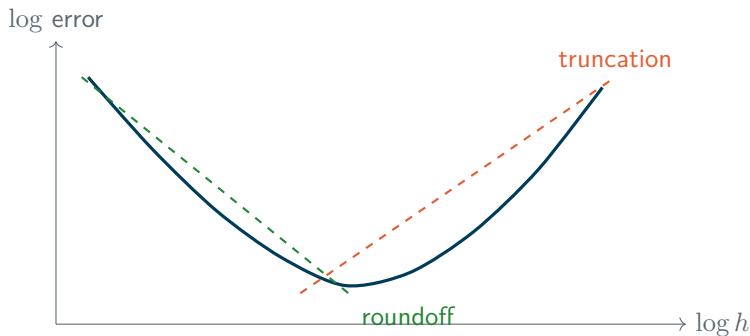
```
def finite_difference_grad(f, x, h=1e-5):
    x = np.asarray(x, dtype=float)
    g = np.empty_like(x)
    for j in range(x.size):
        step = np.zeros_like(x)
        step[j] = h
        g[j] = (f(x + step) - f(x - step)) / (2 * h)
    return g

def relative_error(g, g_ref):
    return np.linalg.norm(g - g_ref) / (
        1.0 + np.linalg.norm(g) + np.linalg.norm(g_ref)
    )
```

Test at random, non-special points—not only at $x = 0$.

The step-size experiment reveals two sources of error

Run the gradient check with $h \in \{10^{-1}, \dots, 10^{-12}\}$.



Question. Why is the smallest available h not the best choice?

Linear regression: derive before you code

Given $X \in \mathbb{R}^{n \times d}$ and $y \in \mathbb{R}^n$,

$$F(w) = \frac{1}{2n} \|Xw - y\|_2^2.$$

Work in pairs:

- 1 derive $\nabla F(w)$ and $H_F(w)$;
- 2 identify the dimensions of every intermediate vector;
- 3 state a condition for a unique minimizer;
- 4 derive the ridge-regularized gradient.

Expected results:

$$\nabla F(w) = \frac{1}{n} X^\top (Xw - y), \quad H_F(w) = \frac{1}{n} X^\top X.$$

Keep the loss and gradient consistent

```
def mse_loss(w, X, y, l2=0.0):  
    residual = X @ w - y  
    return 0.5 * np.mean(residual**2) + 0.5 * l2 * (w @ w)  
  
def mse_grad(w, X, y, l2=0.0):  
    n = X.shape[0]  
    residual = X @ w - y  
    return X.T @ residual / n + l2 * w  
  
def mse_hessian(X, l2=0.0):  
    n, d = X.shape  
    return X.T @ X / n + l2 * np.eye(d)
```

Question. Which line must change if the loss uses a sum rather than a mean?

Coding task: build a reproducible synthetic problem

```
rng = np.random.default_rng(42)
n, d = 200, 8
X = rng.normal(size=(n, d))
w_true = rng.normal(size=d)
y = X @ w_true + 0.2 * rng.normal(size=n)
w0 = rng.normal(size=d)
```

Tasks:

- ① check `mse_grad` at five random parameter vectors;
- ② report the maximum relative error;
- ③ solve the normal equations with `np.linalg.solve`;
- ④ compare w^* with w_{true} ;
- ⑤ inspect the smallest and largest Hessian eigenvalues.

Solve normal equations without an inverse

Avoid

```
w_star = np.linalg.inv(X.T @ X) @ X.T @ y
```

Prefer

```
A = X.T @ X  
b = X.T @ y  
w_star = np.linalg.solve(A, b)
```

or a least-squares factorization:

```
w_star, *_ = np.linalg.lstsq(X, y, rcond=None)
```

The latter remains meaningful when $X^T X$ is singular or nearly singular.

Experiment: rank deficiency breaks uniqueness

Create a duplicate feature:

```
X_bad = np.column_stack([X, X[:, 0]])  
H_bad = X_bad.T @ X_bad / len(X_bad)  
print(np.linalg.eigvalsh(H_bad))
```

Investigate:

- ① Is the loss still convex?
- ② Is the minimizer unique?
- ③ Can different parameter vectors make identical predictions?
- ④ What changes after adding λI with $\lambda = 10^{-3}$?

Takeaway. Rank deficiency is not only a linear-algebra issue; it changes the geometry of learning.

Logistic loss needs both calculus and stable code

For $y_i \in \{-1, +1\}$,

$$F(w) = \frac{1}{n} \sum_i \log(1 + e^{-y_i x_i^\top w}) + \frac{\lambda}{2} \|w\|^2.$$

```
def logistic_loss(w, X, y, l2=0.0):  
    margin = y * (X @ w)  
    return np.mean(np.logaddexp(0.0, -margin)) + 0.5*l2*(w@w)  
  
def logistic_grad(w, X, y, l2=0.0):  
    margin = y * (X @ w)  
    weights = -y / (1.0 + np.exp(margin))  
    return X.T @ weights / X.shape[0] + l2*w
```

For extreme margins, a stable sigmoid implementation may also be required.

Exercise 3: find four bugs without running the code

```
def broken_grad(w, X, y, l2):  
    margin = y * X @ w  
    weights = y / (1 + np.exp(-margin))  
    return X @ weights + 2 * l2 * w
```

Check:

- ① operator precedence in the margin;
- ② the sign and argument of the sigmoid factor;
- ③ matrix dimensions in the aggregation;
- ④ mean versus sum scaling;
- ⑤ consistency of the regularizer definition.

Some lines contain more than one conceptual error.

Exercise 3: one consistent correction

If the objective contains $\frac{\lambda}{2} \|w\|^2$, then

```
def logistic_grad(w, X, y, l2):  
    margin = y * (X @ w)  
    weights = -y / (1.0 + np.exp(margin))  
    return X.T @ weights / X.shape[0] + l2 * w
```

The gradient must have shape $(d,)$. Now construct a unit test that would fail for each original bug. A single random test is insufficient: include extreme margins, $\lambda = 0$, and a non-square X .

Directional checks scale to large models

For high-dimensional w , choose a random unit vector v and compare

$$\langle \nabla f(w), v \rangle \quad \text{with} \quad \frac{f(w + hv) - f(w - hv)}{2h}.$$

```
def directional_check(f, grad, w, rng, h=1e-5):  
    v = rng.normal(size=w.shape)  
    v /= np.linalg.norm(v)  
    analytic = grad(w) @ v  
    numeric = (f(w + h*v) - f(w - h*v)) / (2*h)  
    return analytic, numeric
```

One check costs two function evaluations instead of $2d$, but multiple random directions are needed.

Mini-lab: build a gradient audit report

For the quadratic, MSE, and logistic objectives, produce a table:

Objective	dimension	h	relative error	status
Quadratic				
MSE				
Logistic				

Requirements:

- at least five random points per objective;
- at least three values of h ;
- one deliberately broken gradient to verify the checker detects it;
- fixed random seed and explicit numerical tolerance.

A robust implementation separates responsibilities

- `value(w, data)` computes only the scalar objective.
- `grad(w, data)` computes only the analytic gradient.
- `finite_difference_grad` is an independent diagnostic.
- `validate_shapes` rejects malformed inputs early.
- tests cover random points and constructed edge cases.
- experiment code records seeds, parameters, and metrics.

Question. Why should the finite-difference checker not reuse intermediate values from the analytic gradient?

Exit ticket: explain these three failures

- ❶ The relative gradient error is 10^{-2} for $h = 10^{-3}$ and 10^{-5} , but not for $h = 10^{-1}$. What kind of bug is likely?
- ❷ The gradient passes at $w = 0$ but fails at random w . Why is testing only at zero dangerous?
- ❸ The analytic and numerical gradients differ by a constant factor n . Where should you look first?

Takeaway. Most gradient bugs are not advanced calculus errors—they are signs, shapes, scaling, or unstable formulas.

Five questions every objective must answer

- ① What mathematical objective is implemented?
- ② What are the shapes of every variable and output?
- ③ Does the gradient agree with finite differences?
- ④ What does the Hessian say about local geometry?
- ⑤ Does the implementation remain stable on extreme inputs?

Next seminar: we will turn trusted derivatives into an iterative optimization algorithm.

Sources and further reading

- A. Ignatov, *Optimization Methods: Lecture 1*, HSE University, 2025–2026.
- D. Merkulov, *Optimization for ML, Seminars 1–2*, HSE University, 2025.
<https://hse25.fmin.xyz/program.html>
- S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.
- NumPy documentation: linear algebra, broadcasting, and stable elementary functions.
<https://numpy.org/doc/>